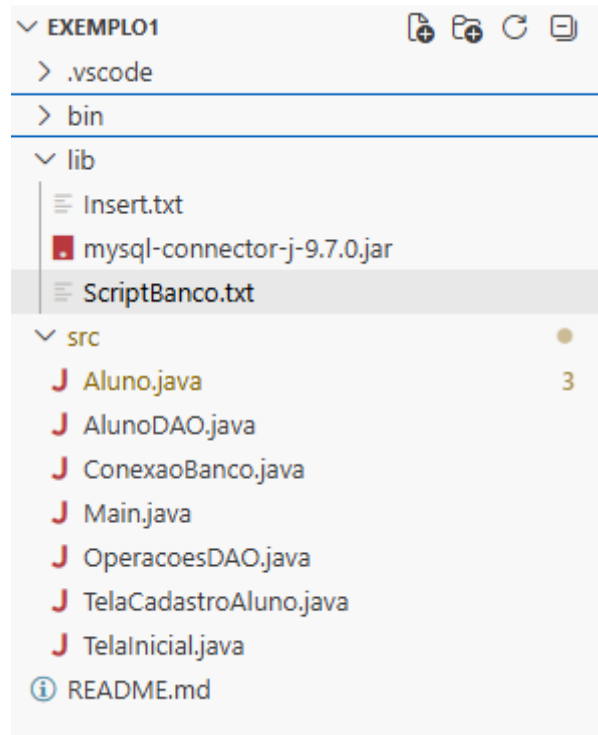


Link GPT: <https://chatgpt.com/g/g-6a280628b9e48191b375b339ac27a8f0-professorgustavo-programacao-em-banco>

At 1 – Implementar as Classes por meio dos códigos fornecidos.

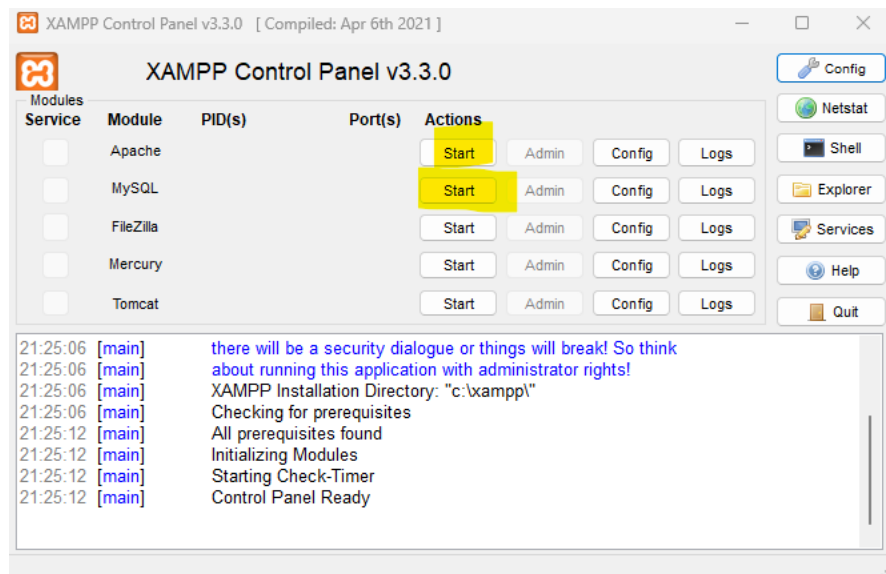
1.1 O resultado desta atividade deve estar semelhante a estrutura apresentada na figura a seguir:



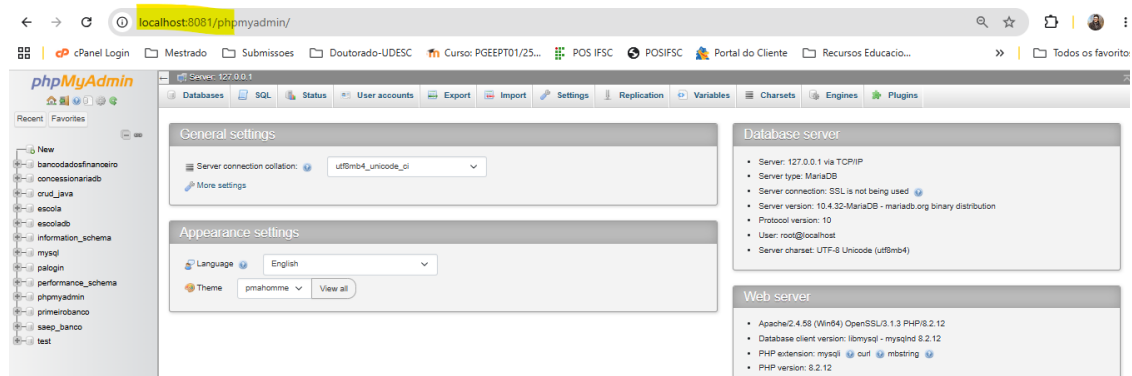
Esta figura ilustra apenas o resultado final esperada. Não é necessária nenhuma intervenção.

1.2 Implementação Banco

Iniciar os serviços do banco



Acessar a URL de acesso ao MySQL. Verifique a porta de conexão caso realizou alguma alteração no arquivo.conf



Na Aba SQL, rode o script de criação do banco e tabela:

```
CREATE DATABASE escola;
```

```
USE escola;
```

```
CREATE TABLE alunos (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```
    nome VARCHAR(100) NOT NULL,
```

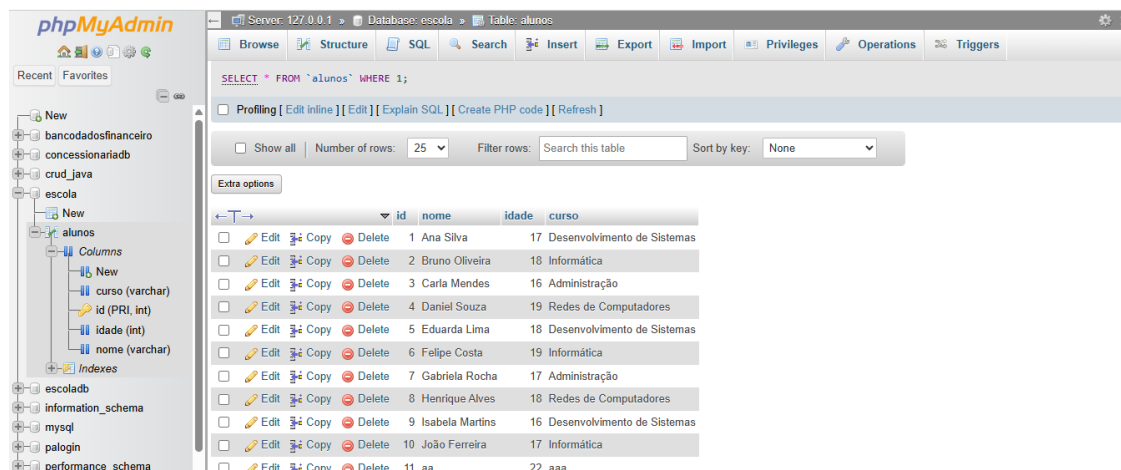
```
    idade INT NOT NULL,
```

```
curso VARCHAR(100) NOT NULL  
);
```

Na aba SQL, rode o script de Inserts
USE escola;

```
INSERT INTO alunos (nome, idade, curso) VALUES  
(  
'Ana Silva', 17, 'Desenvolvimento de Sistemas'),  
(  
'Bruno Oliveira', 18, 'Informática'),  
(  
'Carla Mendes', 16, 'Administração'),  
(  
'Daniel Souza', 17, 'Redes de Computadores'),  
(  
'Eduarda Lima', 18, 'Desenvolvimento de Sistemas'),  
(  
'Felipe Costa', 19, 'Informática'),  
(  
'Gabriela Rocha', 17, 'Administração'),  
(  
'Henrique Alves', 18, 'Redes de Computadores'),  
(  
'Isabela Martins', 16, 'Desenvolvimento de Sistemas'),  
(  
'João Ferreira', 17, 'Informática');
```

Verifique se o banco ficou desta forma



The screenshot shows the phpMyAdmin interface. On the left, the database structure is visible, with 'escola' selected and 'alunos' highlighted. The main panel shows the 'alunos' table structure with columns: id (PRIMARY, INT), nome (VARCHAR), idade (INT), and curso (VARCHAR). Below the structure, the 'Table' tab displays the data rows. The table contains 11 rows of data, with the last row being a placeholder 'aa'.

id	nome	idade	curso
1	Ana Silva	17	Desenvolvimento de Sistemas
2	Bruno Oliveira	18	Informática
3	Carla Mendes	16	Administração
4	Daniel Souza	19	Redes de Computadores
5	Eduarda Lima	18	Desenvolvimento de Sistemas
6	Felipe Costa	19	Informática
7	Gabriela Rocha	17	Administração
8	Henrique Alves	18	Redes de Computadores
9	Isabela Martins	16	Desenvolvimento de Sistemas
10	João Ferreira	17	Informática
11	aa	22	aaa

2.1 Implementação projeto JAVA.

2.1.1 Implementação da classe TelaInicial.java

```
import java.awt.Font;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;

public class TelaInicial extends JFrame {

    public TelaInicial() {

        setTitle("Sistema Escolar");
        setSize(350, 220);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(null);

        JLabel titulo = new JLabel("Sistema de Alunos");
        titulo.setFont(new Font("Arial", Font.BOLD, 20));
        titulo.setBounds(75, 30, 250, 30);
        add(titulo);

        JButton btnCadastrar = new JButton("Gerenciar
Alunos");
        btnCadastrar.setBounds(90, 90, 160, 40);
        add(btnCadastrar);

        btnCadastrar.addActionListener(e -> {
            TelaCadastroAluno telaCadastro = new
TelaCadastroAluno();
            telaCadastro.setVisible(true);
        });
    }
}
```

```
}  
}
```

2.1.2 Implementação da classe TelaCadastroAluno.java

```
import java.util.ArrayList;  
import javax.swing.JButton;  
import javax.swing.JFrame;  
import javax.swing.JLabel;  
import javax.swing.JOptionPane;  
import javax.swing.JScrollPane;  
import javax.swing.JTable;  
import javax.swing.JTextField;  
import javax.swing.table.DefaultTableModel;  
  
public class TelaCadastroAluno extends JFrame {  
  
    private JTextField campoId;  
    private JTextField campoNome;  
    private JTextField campoIdade;  
    private JTextField campoCurso;  
  
    private JTable tabela;  
    private DefaultTableModel modeloTabela;  
  
    public TelaCadastroAluno() {  
  
        setTitle("Cadastro de Aluno - CRUD");  
        setSize(750, 550);  
    }  
}
```

```
setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);  
setLocationRelativeTo(null);  
setLayout(null);
```

```
JLabel labelId = new JLabel("ID:");  
labelId.setBounds(30, 30, 100, 25);  
add(labelId);
```

```
campoId = new JTextField();  
campoId.setBounds(100, 30, 80, 25);  
campoId.setEditable(false);  
add(campoId);
```

```
JLabel labelNome = new JLabel("Nome:");  
labelNome.setBounds(30, 70, 100, 25);  
add(labelNome);
```

```
campoNome = new JTextField();  
campoNome.setBounds(100, 70, 250, 25);  
add(campoNome);
```

```
JLabel labelIdade = new JLabel("Idade:");  
labelIdade.setBounds(30, 110, 100, 25);  
add(labelIdade);
```

```
campoIdade = new JTextField();  
campoIdade.setBounds(100, 110, 250, 25);  
add(campoIdade);
```

```
JLabel labelCurso = new JLabel("Curso:");
```

```
labelCurso.setBounds(30, 150, 100, 25);
add(labelCurso);

campoCurso = new JTextField();
campoCurso.setBounds(100, 150, 250, 25);
add(campoCurso);

JButton btnSalvar = new JButton("Salvar");
btnSalvar.setBounds(390, 30, 120, 35);
add(btnSalvar);

JButton btnAtualizar = new JButton("Atualizar");
btnAtualizar.setBounds(390, 75, 120, 35);
add(btnAtualizar);

JButton btnExcluir = new JButton("Excluir");
btnExcluir.setBounds(390, 120, 120, 35);
add(btnExcluir);

JButton btnLimpar = new JButton("Limpar");
btnLimpar.setBounds(390, 165, 120, 35);
add(btnLimpar);

modeloTabela = new DefaultTableModel();

modeloTabela.addColumn("ID");
modeloTabela.addColumn("Nome");
modeloTabela.addColumn("Idade");
modeloTabela.addColumn("Curso");
```

```

tabela = new JTable(modeloTabela);

JScrollPane scroll = new JScrollPane(tabela);
scroll.setBounds(30, 240, 670, 230);
add(scroll);

carregarTabela();

btnSalvar.addActionListener(e -> cadastrarAluno());

btnAtualizar.addActionListener(e -> atualizarAluno());

btnExcluir.addActionListener(e -> excluirAluno());

btnLimpar.addActionListener(e -> limparCampos());

tabela.addMouseListener(new
java.awt.event.MouseAdapter() {
    public void mouseClicked(java.awt.event.MouseEvent
evt) {
        preencherCampos();
    }
});

private void cadastrarAluno() {

    String nome = campoNome.getText();
    String idadeTexto = campoIdade.getText();
    String curso = campoCurso.getText();

```



```
        if (nome.isEmpty() || idadeTexto.isEmpty() ||
curso.isEmpty()) {
            JOptionPane.showMessageDialog(null, "Preencha
todos os campos!");
            return;
        }

        try {
            int idade = Integer.parseInt(idadeTexto);

            Aluno aluno = new Aluno(nome, idade, curso);

            AlunoDAO dao = new AlunoDAO();
            dao.inserir(aluno);

            JOptionPane.showMessageDialog(null, "Aluno
cadastrado com sucesso!");

            limparCampos();
            carregarTabela();

        } catch (NumberFormatException erro) {
            JOptionPane.showMessageDialog(null, "Idade deve
ser numérica!");
        }
    }

    private void atualizarAluno() {

        if (campoId.getText().isEmpty()) {
```

```
        JOptionPane.showMessageDialog(null, "Selecione um
aluno na tabela!");
        return;
    }

    String nome = campoNome.getText();
    String idadeTexto = campoIdade.getText();
    String curso = campoCurso.getText();

    if (nome.isEmpty() || idadeTexto.isEmpty() ||
curso.isEmpty()) {
        JOptionPane.showMessageDialog(null, "Preencha
todos os campos!");
        return;
    }

    try {
        int id = Integer.parseInt(campoId.getText());
        int idade = Integer.parseInt(idadeTexto);

        Aluno aluno = new Aluno(id, nome, idade, curso);

        AlunoDAO dao = new AlunoDAO();
        dao.atualizar(aluno);

        JOptionPane.showMessageDialog(null, "Aluno
atualizado com sucesso!");

        limparCampos();
        carregarTabela();
    }
```

```

        } catch (NumberFormatException erro) {
            JOptionPane.showMessageDialog(null, "Idade deve
ser numérica!");
        }
    }

    private void excluirAluno() {

        if (campoId.getText().isEmpty()) {
            JOptionPane.showMessageDialog(null, "Selecione um
aluno na tabela!");
            return;
        }

        int resposta = JOptionPane.showConfirmDialog(
            null,
            "Deseja realmente excluir este aluno?",
            "Confirmar exclusão",
            JOptionPane.YES_NO_OPTION
        );

        if (resposta == JOptionPane.YES_OPTION) {

            int id = Integer.parseInt(campoId.getText());

            AlunoDAO dao = new AlunoDAO();
            dao.excluir(id);

            JOptionPane.showMessageDialog(null, "Aluno
excluído com sucesso!");
        }
    }
}

```

```
        limparCampos();
        carregarTabela();
    }
}
```

```
private void carregarTabela() {
    modeloTabela.setRowCount(0);
    AlunoDAO dao = new AlunoDAO();
    ArrayList<Aluno> lista = dao.selecionarTodos();
    for (Aluno aluno : lista) {
        modeloTabela.addRow(new Object[]{
            aluno.getId(),
            aluno.getNome(),
            aluno.getIdade(),
            aluno.getCurso()
        });
    }
}
```

```
private void preencherCampos() {
    int linha = tabela.getSelectedRow();
    if (linha != -1) {
        campoId.setText(modeloTabela.getValueAt(linha,
0).toString());
        campoNome.setText(modeloTabela.getValueAt(linha,
1).toString());
        campoIdade.setText(modeloTabela.getValueAt(linha,
2).toString());
        campoCurso.setText(modeloTabela.getValueAt(linha,
3).toString());
    }
}
```

```

    }

    private void limparCampos() {
        campoId.setText("");
        campoNome.setText("");
        campoIdade.setText("");
        campoCurso.setText("");
        tabela.clearSelection();
    }
}

```

2.1.3 Implemente a classe OperacoesDAO.java

```

import java.util.ArrayList;

public interface OperacoesDAO {
    void inserir(Aluno aluno);
    void atualizar(Aluno aluno);
    void excluir(int id);
    ArrayList<Aluno> selecionarTodos();
}

```

2.1.4 Implemente a classe Main.java

```

public class Main {

    public static void main(String[] args) {

        TelaInicial tela = new TelaInicial();
    }
}

```

```

        tela.setVisible(true);
    }
}

```

2.1.5 Implemente a classe ConexaoBanco.java

```

import java.sql.Connection;
import java.sql.DriverManager;

public class ConexaoBanco {
    private static final String URL
="jdbc:mysql://localhost:3306/escola";
    private static final String USUARIO ="root";
    private static final String SENHA ="";
    public static Connection conectar() {
        try {
            Connection conexao
=DriverManager.getConnection(URL,USUARIO,SENHA);
            System.out.println("Conexão realizada!");
            return conexao;
        } catch (Exception erro) {
            System.out.println("Erro na conexão!");
            System.out.println(erro.getMessage());
            return null;
        }
    }
}

```

2.1.6 Implemente a classe AlunoDAO.java

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.util.ArrayList;

public class AlunoDAO implements OperacoesDAO {

    private Connection conexao;

    public AlunoDAO() {
        conexao = ConexaoBanco.conectar();
    }

    @Override
    public void inserir(Aluno aluno) {
        String sql = "INSERT INTO alunos(nome, idade, curso)
VALUES (?, ?, ?)";
        try {
            PreparedStatement comando =
conexao.prepareStatement(sql);
            comando.setString(1, aluno.getNome());
            comando.setInt(2, aluno.getIdade());
            comando.setString(3, aluno.getCurso());
            comando.executeUpdate();

        } catch (Exception erro) {
            System.out.println("Erro ao inserir aluno!");
            System.out.println(erro.getMessage());
        }
    }
}
```

```

@Override

public void atualizar(Aluno aluno) {
    String sql = "UPDATE alunos SET nome = ?, idade = ?,
curso = ? WHERE id = ?";
    try {
        PreparedStatement comando =
conexao.prepareStatement(sql);
        comando.setString(1, aluno.getNome());
        comando.setInt(2, aluno.getIdade());
        comando.setString(3, aluno.getCurso());
        comando.setInt(4, aluno.getId());
        comando.executeUpdate();
    } catch (Exception erro) {
        System.out.println("Erro ao atualizar aluno!");
        System.out.println(erro.getMessage());
    }
}

```

```

@Override

public void excluir(int id) {
    String sql = "DELETE FROM alunos WHERE id = ?";
    try {
        PreparedStatement comando =
conexao.prepareStatement(sql);
        comando.setInt(1, id);
        comando.executeUpdate();
    } catch (Exception erro) {
        System.out.println("Erro ao excluir aluno!");
        System.out.println(erro.getMessage());
    }
}

```



```

@Override
public ArrayList<Aluno> selecionarTodos() {
    ArrayList<Aluno> listaAlunos = new ArrayList<>();
    String sql = "SELECT * FROM alunos";
    try {
        PreparedStatement comando =
conexao.prepareStatement(sql);
        ResultSet resultado = comando.executeQuery();
        while (resultado.next()) {
            Aluno aluno = new Aluno(
                resultado.getInt("id"),
                resultado.getString("nome"),
                resultado.getInt("idade"),
                resultado.getString("curso")
            );

            listaAlunos.add(aluno);
        }
    } catch (Exception erro) {
        System.out.println("Erro ao selecionar alunos!");
        System.out.println(erro.getMessage());
    }
    return listaAlunos;
}
}

```

2.1.7 Implemente a classe Aluno.java

```

public class Aluno {
    private int id;

```

```
private String nome;
private int idade;
private String curso;
public Aluno(String nome, int idade, String curso) {
    this.nome = nome;
    this.idade = idade;
    this.curso = curso;
}
{
    public Aluno(int id, String nome, int idade, String curso)

        this.id = id;
        this.nome = nome;
        this.idade = idade;
        this.curso = curso;
    }
    public int getId() {
        return id;
    }
    public String getNome() {
        return nome;
    }
    public int getIdade() {
        return idade;
    }
    public String getCurso() {
        return curso;
    }
    public void setId(int id) {
        this.id = id;
    }
}
```

}

Fim da Implementação.

ATIVIDADE 1 –

Melhorar a qualidade do sistema. Idade negativa; idade maior que 120; nome vazio; nome com números; curso vazio.

Passo 1 Impedir idade negativa

Onde implementar?

Métodos:

- cadastrarAluno()
- atualizarAluno()

Código a adicionar após converter a idade

```
if (idade < 0) {  
    JOptionPane.showMessageDialog(null,  
        "A idade não pode ser negativa!");  
    return;  
}
```

Passo 2 - Impedir idade maior que 120 anos

Onde implementar?

Após a validação da idade negativa.

Código

```
if (idade > 120) {  
    JOptionPane.showMessageDialog(null,  
        "A idade deve estar entre 0 e 120 anos!");  
}
```

```
    return;  
}
```

Passo 3 - Impedir nome vazio Atualmente o sistema possui: if (nome.isEmpty() || idadeTexto.isEmpty() || curso.isEmpty())

Substitui por :

```
if (nome.trim().isEmpty()) {  
    JOptionPane.showMessageDialog(null,  
        "O nome não pode ficar vazio!");  
    return;  
}
```

Passo 4 - Impedir números no nome

Onde implementar?

Após verificar se o nome está vazio.

Código

```
if (!nome.matches("[A-Za-zÀ-ÿ ]+")) {  
    JOptionPane.showMessageDialog(null,  
        "O nome deve conter apenas letras!");  
    return;  
}
```

Passo 5 - Impedir curso vazio

Onde implementar?

Após validar o nome.

```
if (curso.trim().isEmpty()) {  
    JOptionPane.showMessageDialog(null,
```

```
        "O curso não pode ficar vazio!");  
  
    return;  
  
}
```

Desafio Extra (para alunos mais avançados)

Implementar as validações diretamente na classe `Aluno`, criando um método:

```
public boolean validarDados()
```

ou lançando exceções:

```
throw new IllegalArgumentException(  
    "Idade inválida!"  
);
```

Assim as regras de negócio ficam centralizadas na classe `Aluno` e não apenas na interface gráfica.

GABARITO

`TelaCadastroAluno.java`. Abaixo estão somente os métodos alterados com o passo a passo informado acima

```
private void cadastrarAluno() {  
  
    String nome = campoNome.getText();  
  
    String idadeTexto = campoIdade.getText();  
  
    String curso = campoCurso.getText();  
  
    if (nome.trim().isEmpty()) {  
        JOptionPane.showMessageDialog(null, "O nome não pode ficar vazio!");  
        return;  
    }  
}
```

```
if (!nome.matches("[A-Za-zÀ-ÿ ]+")) {  
    JOptionPane.showMessageDialog(null, "O nome deve conter apenas letras!");  
    return;  
}
```

```
if (idadeTexto.trim().isEmpty()) {  
    JOptionPane.showMessageDialog(null, "A idade não pode ficar vazia!");  
    return;  
}
```

```
if (curso.trim().isEmpty()) {  
    JOptionPane.showMessageDialog(null, "O curso não pode ficar vazio!");  
    return;  
}
```

```
try {  
    int idade = Integer.parseInt(idadeTexto);  
  
    if (idade < 0) {  
        JOptionPane.showMessageDialog(null, "A idade não pode ser negativa!");  
        return;  
    }  
  
    if (idade > 120) {
```

```
        JOptionPane.showMessageDialog(null, "A idade não pode ser maior que 120 anos!");
```

```
        return;
```

```
    }
```

```
    Aluno aluno = new Aluno(nome.trim(), idade, curso.trim());
```

```
    AlunoDAO dao = new AlunoDAO();
```

```
    dao.inserir(aluno);
```

```
    JOptionPane.showMessageDialog(null, "Aluno cadastrado com sucesso!");
```

```
    limparCampos();
```

```
    carregarTabela();
```

```
    } catch (NumberFormatException erro) {
```

```
        JOptionPane.showMessageDialog(null, "Idade deve ser numérica!");
```

```
    }
```

```
}
```

```
private void atualizarAluno() {
```

```
    if (campoId.getText().isEmpty()) {
```

```
        JOptionPane.showMessageDialog(null, "Selecione um aluno na tabela!");
```

```
        return;
```



```
}
```

```
String nome = campoNome.getText();
```

```
String idadeTexto = campoIdade.getText();
```

```
String curso = campoCurso.getText();
```

```
if (nome.trim().isEmpty()) {
```

```
    JOptionPane.showMessageDialog(null, "O nome não pode ficar vazio!");
```

```
    return;
```

```
}
```

```
if (!nome.matches("[A-Za-zÀ-ÿ ]+")) {
```

```
    JOptionPane.showMessageDialog(null, "O nome deve conter apenas letras!");
```

```
    return;
```

```
}
```

```
if (idadeTexto.trim().isEmpty()) {
```

```
    JOptionPane.showMessageDialog(null, "A idade não pode ficar vazia!");
```

```
    return;
```

```
}
```

```
if (curso.trim().isEmpty()) {
```

```
    JOptionPane.showMessageDialog(null, "O curso não pode ficar vazio!");
```

```
    return;
```

```
}
```

```
try {

    int id = Integer.parseInt(campoId.getText());

    int idade = Integer.parseInt(idadeTexto);

    if (idade < 0) {

        JOptionPane.showMessageDialog(null, "A idade não pode ser negativa!");

        return;

    }

    if (idade > 120) {

        JOptionPane.showMessageDialog(null, "A idade não pode ser maior que 120 anos!");

        return;

    }

    Aluno aluno = new Aluno(id, nome.trim(), idade, curso.trim());

    AlunoDAO dao = new AlunoDAO();

    dao.atualizar(aluno);

    JOptionPane.showMessageDialog(null, "Aluno atualizado com sucesso!");

    limparCampos();

    carregarTabela();

}
```

```
} catch (NumberFormatException erro) {  
  
    JOptionPane.showMessageDialog(null, "Idade deve ser numérica!");  
  
}  
  
}
```

Atividade 2

- Permitir ordenar alunos. Ordenar por Nome Ordenar por Idade.

Objetivo da melhoria Adicionar dois botões: Ordenar por Nome e Ordenar por Idade

Passo 1 Criar métodos no DAO. Classe: AlunoDAO

```
public ArrayList<Aluno> selecionarOrdenadoPorNome() {  
  
    ArrayList<Aluno> listaAlunos = new ArrayList<>();  
  
    String sql = "SELECT * FROM alunos ORDER BY nome ASC";  
  
    try {  
        PreparedStatement comando = conexao.prepareStatement(sql);  
        ResultSet resultado = comando.executeQuery();  
  
        while (resultado.next()) {  
  
            Aluno aluno = new Aluno(  
                resultado.getInt("id"),  
                resultado.getString("nome"),  
                resultado.getInt("idade"),  
                resultado.getString("curso")  
            );  
  
            listaAlunos.add(aluno);  
        }  
    }  
}
```

```

    }

    } catch (Exception erro) {
        System.out.println("Erro ao ordenar por nome!");
    }

    return listaAlunos;
}

```

Adicionar também

```

public ArrayList<Aluno> selecionarOrdenadoPorIdade() {

    ArrayList<Aluno> listaAlunos = new ArrayList<>();

    String sql = "SELECT * FROM alunos ORDER BY idade ASC";

    try {
        PreparedStatement comando = conexao.prepareStatement(sql);
        ResultSet resultado = comando.executeQuery();

        while (resultado.next()) {

            Aluno aluno = new Aluno(
                resultado.getInt("id"),
                resultado.getString("nome"),
                resultado.getInt("idade"),
                resultado.getString("curso")
            );

            listaAlunos.add(aluno);
        }

    } catch (Exception erro) {

```

```

        System.out.println("Erro ao ordenar por idade!");
    }

    return listaAlunos;
}

```

Passo 2 - Classe: TelaCadastroAluno. Criar os botões.

```

JButton btnOrdenarNome = new JButton("Ordenar Nome");
btnOrdenarNome.setBounds(540, 30, 140, 35);
add(btnOrdenarNome);

JButton btnOrdenarIdade = new JButton("Ordenar Idade");
btnOrdenarIdade.setBounds(540, 75, 140, 35);
add(btnOrdenarIdade);

```

Passo 3 - Associar eventos aos botões. Após os outros ActionListener:

```

btnOrdenarNome.addActionListener(e -> carregarTabelaPorNome());
btnOrdenarIdade.addActionListener(e -> carregarTabelaPorIdade());

```

Passo 4 - Criar método para ordenar por nome:

```

private void carregarTabelaPorNome() {

    modeloTabela.setRowCount(0);

    AlunoDAO dao = new AlunoDAO();

    ArrayList<Aluno> lista =
        dao.selecionarOrdenadoPorNome();

    for (Aluno aluno : lista) {

        modeloTabela.addRow(new Object[]{

```

```

        aluno.getId(),
        aluno.getNome(),
        aluno.getIdade(),
        aluno.getCurso()
    });
}
}

```

Passo 5 - Criar método para ordenar por idade

```

private void carregarTabelaPorIdade() {

    modeloTabela.setRowCount(0);

    AlunoDAO dao = new AlunoDAO();

    ArrayList<Aluno> lista =
        dao.selecionarOrdenadoPorIdade();

    for (Aluno aluno : lista) {

        modeloTabela.addRow(new Object[]{
            aluno.getId(),
            aluno.getNome(),
            aluno.getIdade(),
            aluno.getCurso()
        });
    }
}

```

GABARITO

AlunoDAO

```

public ArrayList<Aluno> selecionarOrdenadoPorNome() {

    ArrayList<Aluno> listaAlunos = new ArrayList<>();

    String sql = "SELECT * FROM alunos ORDER BY nome ASC";

    try {
        PreparedStatement comando = conexao.prepareStatement(sql);
        ResultSet resultado = comando.executeQuery();

        while (resultado.next()) {
            Aluno aluno = new Aluno(
                resultado.getInt("id"),
                resultado.getString("nome"),
                resultado.getInt("idade"),
                resultado.getString("curso")
            );

            listaAlunos.add(aluno);
        }

    } catch (Exception erro) {
        System.out.println("Erro ao ordenar alunos por nome!");
        System.out.println(erro.getMessage());
    }

    return listaAlunos;
}

public ArrayList<Aluno> selecionarOrdenadoPorIdade() {

    ArrayList<Aluno> listaAlunos = new ArrayList<>();

    String sql = "SELECT * FROM alunos ORDER BY idade ASC";

```

```

try {
    PreparedStatement comando = conexao.prepareStatement(sql);
    ResultSet resultado = comando.executeQuery();

    while (resultado.next()) {
        Aluno aluno = new Aluno(
            resultado.getInt("id"),
            resultado.getString("nome"),
            resultado.getInt("idade"),
            resultado.getString("curso")
        );

        listaAlunos.add(aluno);
    }

} catch (Exception erro) {
    System.out.println("Erro ao ordenar alunos por idade!");
    System.out.println(erro.getMessage());
}

return listaAlunos;
}

```

TelaCadastroAluno

```

import java.util.ArrayList;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.table.DefaultTableModel;

public class TelaCadastroAluno extends JFrame {

```



```
private JTextField campoId;
private JTextField campoNome;
private JTextField campoIdade;
private JTextField campoCurso;

private JTable tabela;
private DefaultTableModel modeloTabela;

public TelaCadastroAluno() {

    setTitle("Cadastro de Aluno - CRUD");
    setSize(750, 550);
    setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(null);

    JLabel labelId = new JLabel("ID:");
    labelId.setBounds(30, 30, 100, 25);
    add(labelId);

    campoId = new JTextField();
    campoId.setBounds(100, 30, 80, 25);
    campoId.setEditable(false);
    add(campoId);

    JLabel labelNome = new JLabel("Nome:");
    labelNome.setBounds(30, 70, 100, 25);
    add(labelNome);

    campoNome = new JTextField();
    campoNome.setBounds(100, 70, 250, 25);
    add(campoNome);

    JLabel labelIdade = new JLabel("Idade:");
    labelIdade.setBounds(30, 110, 100, 25);
    add(labelIdade);

    campoIdade = new JTextField();
    campoIdade.setBounds(100, 110, 250, 25);
    add(campoIdade);

    JLabel labelCurso = new JLabel("Curso:");
    labelCurso.setBounds(30, 150, 100, 25);
    add(labelCurso);

    campoCurso = new JTextField();
    campoCurso.setBounds(100, 150, 250, 25);
    add(campoCurso);

    JButton btnSalvar = new JButton("Salvar");
    btnSalvar.setBounds(390, 30, 120, 35);
    add(btnSalvar);

    JButton btnAtualizar = new JButton("Atualizar");
    btnAtualizar.setBounds(390, 75, 120, 35);
    add(btnAtualizar);

    JButton btnExcluir = new JButton("Excluir");
    btnExcluir.setBounds(390, 120, 120, 35);
```

```

        add(btnExcluir);

        JButton btnLimpar = new JButton("Limpar");
        btnLimpar.setBounds(390, 165, 120, 35);
        add(btnLimpar);

        JButton btnOrdenarNome = new JButton("Ordenar Nome");
        btnOrdenarNome.setBounds(540, 30, 140, 35);
        add(btnOrdenarNome);

        JButton btnOrdenarIdade = new JButton("Ordenar Idade");
        btnOrdenarIdade.setBounds(540, 75, 140, 35);
        add(btnOrdenarIdade);

        modeloTabela = new DefaultTableModel();

        modeloTabela.addColumn("ID");
        modeloTabela.addColumn("Nome");
        modeloTabela.addColumn("Idade");
        modeloTabela.addColumn("Curso");

        tabela = new JTable(modeloTabela);

        JScrollPane scroll = new JScrollPane(tabela);
        scroll.setBounds(30, 240, 670, 230);
        add(scroll);

        carregarTabela();

        btnSalvar.addActionListener(e -> cadastrarAluno());
        btnAtualizar.addActionListener(e -> atualizarAluno());
        btnExcluir.addActionListener(e -> excluirAluno());
        btnLimpar.addActionListener(e -> limparCampos());

        btnOrdenarNome.addActionListener(e -> carregarTabelaPorNome());
        btnOrdenarIdade.addActionListener(e -> carregarTabelaPorIdade());

        tabela.addMouseListener(new java.awt.event.MouseAdapter() {
            public void mouseClicked(java.awt.event.MouseEvent evt) {
                preencherCampos();
            }
        });
    }

    private void cadastrarAluno() {

        String nome = campoNome.getText();
        String idadeTexto = campoIdade.getText();
        String curso = campoCurso.getText();

        if (nome.trim().isEmpty()) {
            JOptionPane.showMessageDialog(null, "O nome não pode ficar
vazio!");
            return;
        }

        if (!nome.matches("[A-Za-zÀ-ÿ ]+")) {
            JOptionPane.showMessageDialog(null, "O nome deve conter apenas
letras!");
        }
    }

```

```

        return;
    }

    if (idadeTexto.trim().isEmpty()) {
        JOptionPane.showMessageDialog(null, "A idade não pode ficar vazia!");
        return;
    }

    if (curso.trim().isEmpty()) {
        JOptionPane.showMessageDialog(null, "O curso não pode ficar vazio!");
        return;
    }

    try {
        int idade = Integer.parseInt(idadeTexto);

        if (idade < 0) {
            JOptionPane.showMessageDialog(null, "A idade não pode ser negativa!");
            return;
        }

        if (idade > 120) {
            JOptionPane.showMessageDialog(null, "A idade não pode ser maior que 120 anos!");
            return;
        }

        Aluno aluno = new Aluno(nome.trim(), idade, curso.trim());

        AlunoDAO dao = new AlunoDAO();
        dao.inserir(aluno);

        JOptionPane.showMessageDialog(null, "Aluno cadastrado com sucesso!");

        limparCampos();
        carregarTabela();

    } catch (NumberFormatException erro) {
        JOptionPane.showMessageDialog(null, "Idade deve ser numérica!");
    }
}

private void atualizarAluno() {

    if (campoId.getText().isEmpty()) {
        JOptionPane.showMessageDialog(null, "Selecione um aluno na tabela!");
        return;
    }

    String nome = campoNome.getText();
    String idadeTexto = campoIdade.getText();
    String curso = campoCurso.getText();

```

```

        if (nome.trim().isEmpty()) {
            JOptionPane.showMessageDialog(null, "O nome não pode ficar
vazio!");
            return;
        }

        if (!nome.matches("[A-Za-zÀ-ÿ ]+")) {
            JOptionPane.showMessageDialog(null, "O nome deve conter apenas
letras!");
            return;
        }

        if (idadeTexto.trim().isEmpty()) {
            JOptionPane.showMessageDialog(null, "A idade não pode ficar
vazia!");
            return;
        }

        if (curso.trim().isEmpty()) {
            JOptionPane.showMessageDialog(null, "O curso não pode ficar
vazio!");
            return;
        }

        try {
            int id = Integer.parseInt(campoId.getText());
            int idade = Integer.parseInt(idadeTexto);

            if (idade < 0) {
                JOptionPane.showMessageDialog(null, "A idade não pode ser
negativa!");
                return;
            }

            if (idade > 120) {
                JOptionPane.showMessageDialog(null, "A idade não pode ser
maior que 120 anos!");
                return;
            }

            Aluno aluno = new Aluno(id, nome.trim(), idade, curso.trim());

            AlunoDAO dao = new AlunoDAO();
            dao.atualizar(aluno);

            JOptionPane.showMessageDialog(null, "Aluno atualizado com
sucesso!");

            limparCampos();
            carregarTabela();

        } catch (NumberFormatException erro) {
            JOptionPane.showMessageDialog(null, "Idade deve ser
numérica!");
        }
    }

    private void excluirAluno() {

```

```

        if (campoId.getText().isEmpty()) {
            JOptionPane.showMessageDialog(null, "Selecione um aluno na
tabela!");
            return;
        }

        int resposta = JOptionPane.showConfirmDialog(
            null,
            "Deseja realmente excluir este aluno?",
            "Confirmar exclusão",
            JOptionPane.YES_NO_OPTION
        );

        if (resposta == JOptionPane.YES_OPTION) {

            int id = Integer.parseInt(campoId.getText());

            AlunoDAO dao = new AlunoDAO();
            dao.excluir(id);

            JOptionPane.showMessageDialog(null, "Aluno excluído com
sucesso!");

            limparCampos();
            carregarTabela();
        }
    }

    private void carregarTabela() {

        modeloTabela.setRowCount(0);

        AlunoDAO dao = new AlunoDAO();
        ArrayList<Aluno> lista = dao.selecionarTodos();

        for (Aluno aluno : lista) {
            modeloTabela.addRow(new Object[]{
                aluno.getId(),
                aluno.getNome(),
                aluno.getIdade(),
                aluno.getCurso()
            });
        }
    }

    private void carregarTabelaPorNome() {

        modeloTabela.setRowCount(0);

        AlunoDAO dao = new AlunoDAO();
        ArrayList<Aluno> lista = dao.selecionarOrdenadoPorNome();

        for (Aluno aluno : lista) {
            modeloTabela.addRow(new Object[]{
                aluno.getId(),
                aluno.getNome(),
                aluno.getIdade(),
                aluno.getCurso()
            });
        }
    }

```

```

    }
}

private void carregarTabelaPorIdade() {

    modeloTabela.setRowCount(0);

    AlunoDAO dao = new AlunoDAO();
    ArrayList<Aluno> lista = dao.selecionarOrdenadoPorIdade();

    for (Aluno aluno : lista) {
        modeloTabela.addRow(new Object[]{
            aluno.getId(),
            aluno.getNome(),
            aluno.getIdade(),
            aluno.getCurso()
        });
    }
}

private void preencherCampos() {

    int linha = tabela.getSelectedRow();

    if (linha != -1) {
        campoId.setText(modeloTabela.getValueAt(linha, 0).toString());
        campoNome.setText(modeloTabela.getValueAt(linha,
1).toString());
        campoIdade.setText(modeloTabela.getValueAt(linha,
2).toString());
        campoCurso.setText(modeloTabela.getValueAt(linha,
3).toString());
    }
}

private void limparCampos() {

    campoId.setText("");
    campoNome.setText("");
    campoIdade.setText("");
    campoCurso.setText("");
    tabela.clearSelection();
}
}

```

Atividade 3

Implementar pesquisa de alunos por nome. possuir um campo de pesquisa; possuir botão Buscar; mostrar apenas os alunos encontrados.

Passo 1 – Criar método de busca no DAO

Classe: AlunoDAO

Adicionar o método:

```
public ArrayList<Aluno> pesquisarPorNome(String nomePesquisa) {  
  
    ArrayList<Aluno> listaAlunos = new ArrayList<>();  
  
    String sql = "SELECT * FROM alunos WHERE nome LIKE ?";  
  
    try {  
  
        PreparedStatement comando = conexao.prepareStatement(sql);  
  
        comando.setString(1, "%" + nomePesquisa + "%");  
  
        ResultSet resultado = comando.executeQuery();  
  
        while (resultado.next()) {  
  
            Aluno aluno = new Aluno(  
                resultado.getInt("id"),  
                resultado.getString("nome"),  
                resultado.getInt("idade"),  
                resultado.getString("curso")  
            );  
  
            listaAlunos.add(aluno);  
  
        }  
  
    } catch (Exception erro) {  
        System.out.println("Erro ao pesquisar alunos!");  
        System.out.println(erro.getMessage());  
    }  
  
    return listaAlunos;  
}
```

Passo 2 – Criar campo de pesquisa

Classe: TelaCadastroAluno

Adicionar atributo:

```
private JTextField campoPesquisa;
```

Adicionar componentes no constructor

```
JLabel labelPesquisa = new JLabel("Pesquisar:");
labelPesquisa.setBounds(30, 200, 80, 25);
add(labelPesquisa);

campoPesquisa = new JTextField();
campoPesquisa.setBounds(110, 200, 240, 25);
add(campoPesquisa);

JButton btnBuscar = new JButton("Buscar");
btnBuscar.setBounds(370, 200, 120, 25);
add(btnBuscar);
```

Passo 3 – Evento do botão Buscar

Adicionar no construtor:

```
btnBuscar.addActionListener(e -> pesquisarAluno());
```

Passo 4 – Criar método pesquisarAluno(). Adicionar na classe:

```
private void pesquisarAluno() {

    String nomePesquisa = campoPesquisa.getText();

    if (nomePesquisa.trim().isEmpty()) {

        JOptionPane.showMessageDialog(
            null,
            "Digite um nome para pesquisar!"
        );

        return;
    }

    modeloTabela.setRowCount(0);

    AlunoDAO dao = new AlunoDAO();

    ArrayList<Aluno> lista =
        dao.pesquisarPorNome(nomePesquisa);

    if (lista.isEmpty()) {

        JOptionPane.showMessageDialog(
            null,
            "Nenhum aluno encontrado!"
        );
    }
}
```



```
    );  
  
    return;  
}  
  
for (Aluno aluno : lista) {  
  
    modeloTabela.addRow(new Object[]{  
        aluno.getId(),  
        aluno.getNome(),  
        aluno.getIdade(),  
        aluno.getCurso()  
    });  
}  
}
```